

ABHIMANYU X

Autonomous Cyber Immune System for Defence Software

THE PROBLEM

- › **AI patch tools trust the LLM.** — a generated fix is shown as done, with no proof it actually blocks the exploit.
- › **Security tooling needs heavy infra.** — cloud APIs, GPUs, or clusters most defence-context labs can't rely on.
- › **Manual discover-patch-verify cycles are slow** — and don't get faster just by adding an LLM in the loop.

THE IDEA

- › **A closed-loop cyber-immune cell.** — finds a real vulnerability, reasons about root cause with a local LLM, patches it, and PROVES the fix holds.
- › **AI proposes. Evidence decides.** — every patch must pass real compiler + real exploit replay + real regression before it's called "verified."
- › **Fully local inference.** — a 3B-parameter model running offline — no cloud dependency, air-gap capable.

CORE LOOP



Every verified vulnerability becomes future defence.

Detailed Methodology

Step-by-step pipeline — every stage below actually executes

1. REWIND

Real `git diff` commit analysis + static detection engine (Python + C). Includes a custom rule the project added: catching unchecked length reaching memcpy() — CWE-120 — which no pre-existing pattern covered.

3. ANVIL — AI Reasoning

Root-cause analysis and patch generation via a local Ollama model (Qwen2.5-Coder 3B). No cloud call, no chain-of-thought exposed — only structured, evidence-grounded conclusions.

5. Self-Correcting Repair

A failed verification sends the concrete failure reason back to ANVIL for a real second attempt — GENERATE → TEST → FAIL → REPAIR → RETEST, not one-shot patching.

2. Dynamic Analysis & Fuzzing

AFL++ 4.09c installed as a genuine OS package inside a Linux VM (Colima/Ubuntu 24.04). Blind-mode fuzzing against a real clang+ASan binary found a real stack-buffer-overflow crash. A real LLVM17/ASan toolchain incompatibility was found and disclosed, not hidden.

4. Verification Gates

Real compile check, real exploit replay (same crash input, before vs. after, recompiled and re-executed in the VM), real regression/behaviour checks, plus adversarial testing across 6 additional payload sizes spanning the boundary.

6. Immune Memory & Transfer

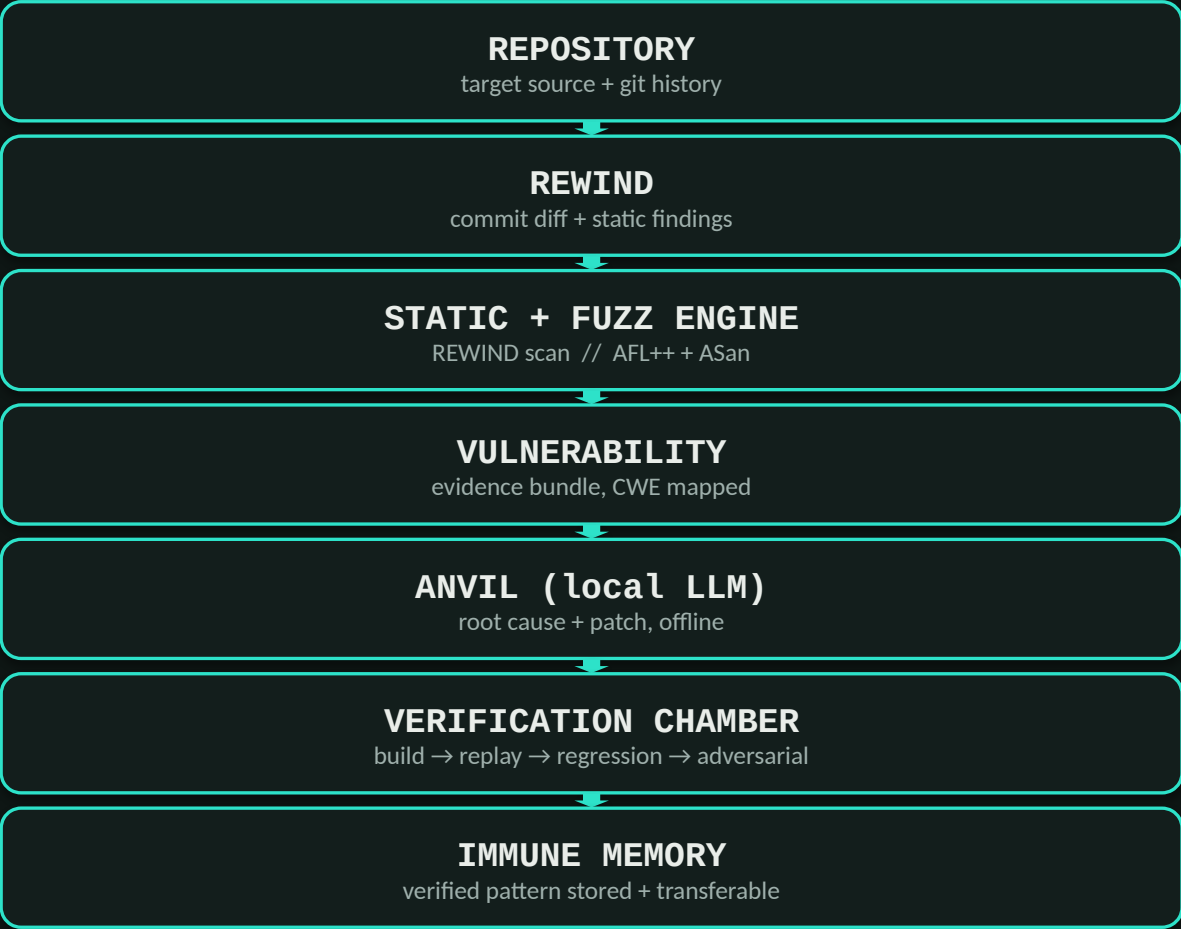
Verified patterns are stored as “Vulnerability DNA” in SQLite. A second real target (network_protocol_parser) demonstrates a measured with-memory vs. without-memory comparison — not a fabricated benchmark.

Technology Stack & System Flow

STACK

- › **Static analysis** — REWIND engine — Python, regex + AST + heuristics
- › **Fuzzing / dynamic** — AFL++ 4.09c, clang-18, AddressSanitizer
- › **Local AI** — Ollama + Qwen2.5-Coder 3B — fully offline inference
- › **Backend** — Python, Flask + Flask-SocketIO (real-time WebSocket events)
- › **Data** — SQLite — Immune Memory / Vulnerability DNA store
- › **Frontend** — Vanilla Three.js 3D laboratory — no build toolchain, reacts to real backend state
- › **CLI** — Python argparse — doctor / scan / mission / transfer
- › **Supply chain** — pip-audit (real SBOM/CVE scan), Docker buildx (real multi-arch check)
- › **Equipment** — MacBook (Apple Silicon) host + Colima-managed Linux VM — 2 vCPU / 4 GB RAM / 20 GB disk

SYSTEM FLOW



What Makes This Different

AI proposes. Evidence decides. Verification proves. Memory learns.

Evidence-driven verification, not AI self-certification

A patch is only called “VERIFIED” after passing all real gates — compile, exploit replay, regression, behaviour, adversarial robustness (7/7, computed, never asserted).

Measured Immune Transfer, not a marketing claim

A second real vulnerable target shows an actual with-memory vs. without-memory comparison — reported as measured, whatever the real result turned out to be.

Radical honesty baked into the UI itself

Every figure is tagged MEASURED / AI-GENERATED / DEMO / FUTURE. A real, disclosed toolchain limitation (AFL++ coverage instrumentation vs. ASan on this platform) is documented rather than hidden — and worked around with a real fix, not a fudge.

Self-correcting repair loop

A rejected patch's real failure reason is sent back to ANVIL for a genuine second attempt — the system demonstrably knows when NOT to trust its own output.

Fully local, fully lightweight

A 3B-parameter model and a 2-vCPU/4GB VM run the entire discover→patch→verify→remember loop on a single laptop — no GPU, no cluster, no cloud API.

Reproducible by construction

Every mission emits real provenance: commit hash, target/patch SHA-256, tool versions, timestamps — the same run can be independently re-verified.

Deliverables & Proof of Concept

DELIVERED

- › Working end-to-end prototype: DISCOVER → UNDERSTAND → REPAIR → VERIFY → REMEMBER → TRANSFER
- › Two real vulnerable C targets — real git history, real fuzz harness, real AFL+/ASan-found crash each
- › 3D interactive laboratory UI (Three.js) and a CLI, both driven by the same real backend event stream
- › Real per-mission Provenance record for reproducibility (commit hash, file/patch SHA-256, tool versions, timestamps)
- › Real SBOM (pip-audit) and real multi-architecture capability report (Docker buildx)
- › Retry/rollback loop and adversarial robustness testing on every verified patch

DEMONSTRATED LIVE

Real crash found (AFL++/ASan)

Real patch generated (local LLM)

Real exploit replay – blocked

Real regression – pass

Immune Memory created

Pattern recognized on 2nd real target

Status: research prototype — not validated for production or defence-classified deployment.

Every patch is meant to be human-reviewed before use, not auto-deployed.